

S3 — SIMPLE STORAGE SERVICE

WHAT IS AMAZON S3?

- Amazon S3 (Simple Storage Service) is an object storage service that lets you store and retrieve any amount of data from anywhere.
- It is designed for 99.999999999% (11 9's) durability and 99.99% availability.
- S3 solves the problem of storing **massive** amounts of **unstructured data** reliably, securely and cost-effectively.

WHAT IS OBJECT STORAGE?

- Data is stored as **OBJECTS** (not files/folders).
- Each object contains: data + metadata + key.
- Objects are stored inside **BUCKETS**.

S3 VS TRADITIONAL HARD DISK

TRADITIONAL HARD DISK	S3 (OBJECT STORAGE)
• Limited capacity • Single point of failure • File/Folder based • Vertically scaled • You manage hardware • Not ideal for massive scale	• Virtually unlimited • Highly durable & available • Object based (key + data) • Horizontally scaled • AWS manages everything • Built for massive scale

WHY IS S3 HIGHLY SCALABLE?

- ✓ Built on distributed architecture across multiple AWS Availability Zones.
- ✓ Automatically scales to handle unlimited data and request volumes.
- ✓ No need to provision capacity.
- ✓ Pay only for what you store and use.



REMEMBER

"S3 stores **OBJECTS** inside **BUCKETS**."



ANALOGY

S3 = A virtually unlimited digital storage room where you store objects.



KEY POINT

You store anything as an **OBJECT**.
S3 takes care of durability, availability and scaling.



COMMON REAL-WORLD USE CASES

	Website Assets	→ HTML, CSS, JS, images, fonts
	Backups	→ Database backups, system backups
	Data Lakes	→ Store large datasets for analytics
	Images / Videos	→ Store and deliver media files
	Logs	→ Application logs, access logs
	Static Websites	→ Host static websites on S3



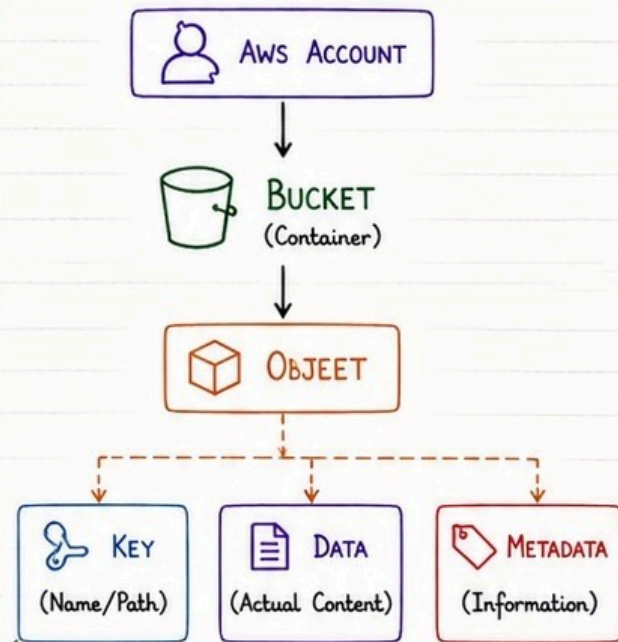
SUMMARY (IN ONE LINE)

Amazon S3 is a durable, secure, highly scalable object storage service that lets you store and retrieve **any amount of data** from anywhere on the internet.



UNDERSTAND THESE BEFORE USING S3

S3 HIERARCHY



EXAMPLE

Bucket: my-company-data
 Object Key: backups/2024/Jan/report.pdf
 Data: (binary content of the file)
 Metadata: size, type, last-modified, etc.

S3 CORE CONCEPTS

- Bucket** - A container that holds objects.
 - Bucket names are globally unique.
 - Buckets live in a specific AWS Region.
- Object** - The fundamental entity stored in S3.
 - Consists of data + metadata + key.
- Object Key** - The unique identifier of an object within a bucket.
 - Example: folder1/folder2/file.txt
 - Keys can look like folders, but S3 is flat.
- Metadata** - Data about the object.
 - System metadata: size, type, last-modified.
 - User metadata: custom key-value pairs.
- Region** - Buckets are created in a specific AWS Region (e.g., us-east-1).
- Bucket ARN** - Unique Amazon Resource Name for a bucket.
Example: arn:aws:s3:::my-bucket
- Object URL** - URL used to access an object.
Example:

`https:// my-bucket.s3.amazonaws.com/folder/file.jpg`

CONCEPTS AT A GLANCE

CONCEPT	MEANING
Bucket	Container for objects. Similar to a top-level folder, but globally unique.
Object	The actual data stored in S3. Every object has data, key, and metadata.
Key	The unique name/path of the object within the bucket.
Metadata	Information about the object (size, type, timestamps, custom data).
Region	Geographic location where your bucket/data is stored.
Bucket ARN	The Amazon Resource Name used for permissions and policies.
Object URL	Web address (if public) or presigned URL to access the object.

★ KEY IDEA

S3 is not a traditional folder-based filesystem.

Folders are represented through object key prefixes.

BUCKET NAMING RULES (QUICK REMINDER)

- 3 to 63 characters long.
- Must be globally unique.
- Lowercase letters, numbers, dots (.), and hyphens (-) only.
- Must start and end with a letter or number.
- Cannot be an IP address (e.g., 192.168.1.1).
- No uppercase letters or underscores.

GOOD EXAMPLES

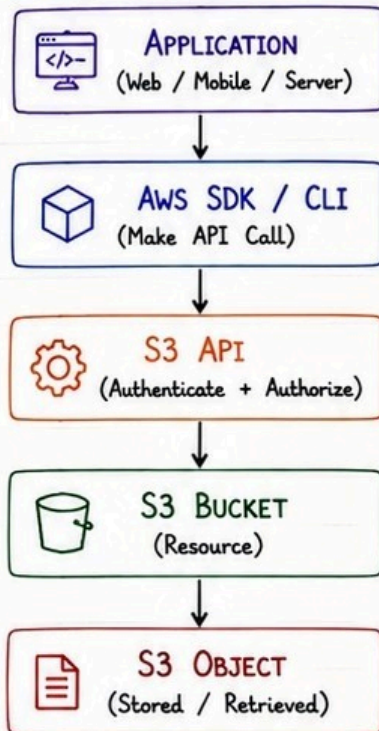
- ✓ my-company-data
- ✓ logs-2024
- ✓ backup.bucket-01

BAD EXAMPLES

- ✗ My_Bucket
- ✗ 192.168.1.1
- ✗ my..bucket
- ✗ -mybucket

How Does AN S3 REQUEST WORK?

S3 REQUEST FLOW (HIGH LEVEL)



BASIC OPERATIONS



PUT Object

Upload an object to S3.
Creates a new object or replaces an existing one.



GET Object

Retrieve an object from S3.
Returns the object data.



DELETE Object

Permanently delete an object.
(If versioning is OFF)



LIST Objects

List objects in a bucket.
Optionally filter by prefix.



KEY IDEA

S3 is a **REST**-based service.
All operations are done using
HTTP requests.



UPLOAD FLOW (PUT OBJECT)



- Data is sent to S3.
- S3 stores the object.
- You get a success response with status code (200 OK).

DOWNLOAD FLOW (GET OBJECT)



- Request is sent to S3.
- S3 fetches the object.
- S3 returns the object data to the user.

S3 COMMON COMMANDS (AWS CLI)

\$		
# List all buckets	<code>aws s3 ls</code>	→ List buckets in your account
# List objects in a bucket	<code>aws s3 ls s3://my-bucket/</code>	→ List objects in a bucket
# Upload a file to S3	<code>aws s3 cp file.txt s3://my-bucket/</code>	→ Upload file to bucket
# Download a file from S3	<code>aws s3 cp s3://my-bucket/file.txt .</code>	→ Download file to current dir
# Sync a local folder to S3	<code>aws s3 sync ./website s3://my-bucket/</code>	→ Sync local folder to S3
# Sync S3 to local folder	<code>aws s3 sync s3://my-bucket/ ./backup/</code>	→ Sync S3 to local folder
# Delete an object	<code>aws s3 rm s3://my-bucket/file.txt</code>	→ Delete single object
# Recursive delete	<code>aws s3 rm s3://my-bucket/ --recursive</code>	→ Delete all objects in bucket



NOTE

- CLI uses your IAM permissions.
- Use "aws configure" to set up credentials.
- Always test with small files first.



COMMON MISTAKE

- Uploading large files without multipart upload can fail or timeout.
- Use "aws s3 cp" (CLI handles multipart automatically).



REMEMBER

All S3 actions are API calls. SDK or CLI is just a layer on top of S3 API.



CHOOSE THE RIGHT STORAGE CLASS

- S3 offers multiple storage classes designed for different access patterns and **cost requirements**.

STORAGE CLASS	ACCESS PATTERN	DURABILITY	AVAILABILITY	MAIN IDEA
 S3 Standard	Frequently accessed data	99.999999999% (11 9's)	99.99%	General purpose. Best choice for most use cases.
 S3 Intelligent-Tiering	Changing or unknown	99.999999999% (11 9's)	99.9%	Automatically moves data to the most cost-effective tier.
 S3 Standard-IA (Infrequent Access)	Infrequently accessed data	99.999999999% (11 9's)	99.9%	Lower cost for data that is accessed less often.
 S3 One Zone-IA (Infrequent Access)	Re-creatable, infrequently accessed	99.999999999% (11 9's)	99.5%	Stores data in a single Availability Zone. Lower cost.
 S3 Glacier Instant Retrieval	Archive data, rarely accessed	99.999999999% (11 9's)	99.9%	Archive data with millisecond retrieval.
 S3 Glacier Flexible Retrieval	Archive data, very rarely accessed	99.999999999% (11 9's)	99.9%	Low cost archive with retrieval in minutes to hours.
 S3 Glacier Deep Archive	Archive data, almost never accessed	99.999999999% (11 9's)	99.9%	Lowest cost storage for long-term archives.

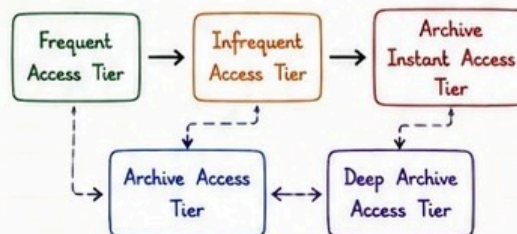


PRO TIP

- Storage class selection is a cost + access-pattern decision.
- There is no "best" class, only the best fit for your data behavior.



HOW S3 INTELLIGENT-TIERING WORKS



S3 automatically moves objects between tiers based on access patterns.

KEY TAKEAWAY

Use the right storage class to optimize **COST** without sacrificing **PERFORMANCE**. Most organizations start with S3 Standard and use Lifecycle rules to move data to cheaper classes over time.



COMPARE AT A GLANCE

STORAGE CLASS	COST	ACCESS FREQUENCY	RETRIEVAL TIME
S3 Standard	\$\$	High	Milliseconds
S3 Intelligent-Tiering	\$\$	Varying / Unknown	Milliseconds
S3 Standard-IA	\$	Low	Milliseconds
S3 One Zone-IA	\$	Low (Re-creatable)	Milliseconds
S3 Glacier Instant Retrieval	\$	Archive (Occasional)	Milliseconds
S3 Glacier Flexible Retrieval	¢	Archive (Rare)	Minutes to Hours
S3 Glacier Deep Archive	¢	Archive (Very Rare)	Hours

\$\$\$ = Highest Cost \$\$ = Medium \$ = Low ¢ = Lowest

WHEN TO USE WHAT?

 S3 Standard	→ Active websites, general workloads
 Intelligent-Tiering	→ Unpredictable workloads
 Standard-IA	→ Backups, disaster recovery
 One Zone-IA	→ Re-creatable data, secondary copies
 Glacier Instant	→ Archives with fast access need
 Glacier Flexible	→ Long-term archive (minutes-hours)
 Glacier Deep	→ Long-term archive (hours)

STORAGE CLASS TRANSITION (AUTOMATED)

UPLOAD



S3 Standard-IA /
One Zone-IA
(Infrequent)

Glacier Classes
(Archive)



DELETE
(Expired)



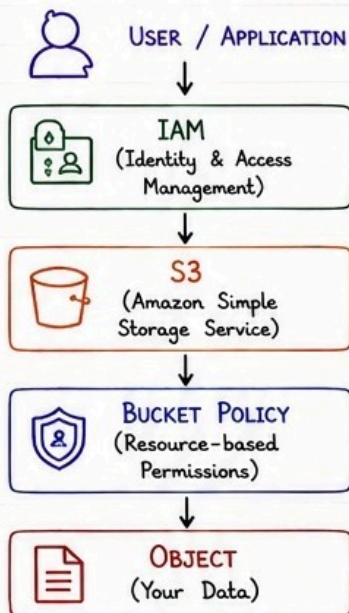
REMEMBER

Your data changes over time. Let **Lifecycle Rules** move it to the right storage class automatically.



WHO CAN ACCESS MY S3 BUCKET?

S3 SECURITY OVERVIEW



KEY POINT

Access to an object is allowed only if **BOTH** the IAM permissions and Bucket policy allow it.

1. IAM POLICIES (IDENTITY-BASED)

- IAM policies are attached to users, groups or roles.
- They define what actions are allowed on which resources.
- Example: Allow a user to read objects from a bucket.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
  
```

This allows the user to GET objects from my-bucket.

2. BUCKET POLICIES (RESOURCE-BASED)

- Bucket policies are attached to the bucket.
- They define who can access the bucket and what actions.
- Useful to allow access from specific AWS accounts or services.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {"AWS": "123456789012"},
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::my-bucket"
    }
  ]
}
  
```

Allows AWS account 123456789012 to list the bucket.

3. BLOCK PUBLIC ACCESS (BPA)

AWS recommends enabling Block Public Access at the account or bucket level.

It helps prevent accidental public exposure.

4 BPA SETTINGS

- ☒ Block public access to buckets and objects granted through new access control lists (ACLs)
- ☒ Block public access to buckets and objects granted through any ACLs
- ☒ Block public access to buckets and objects granted through new public bucket or access point policies
- ☒ Block public and cross-account access to buckets and objects through any public bucket or access point policies



4. ENCRYPTION

- Always enable encryption to protect data at rest.
- S3 offers:

🔑 SSE-S3 (AES-256)

🔑 SSE-KMS (AWS KMS Keys)

🔑 SSE-C (Customer-Provided Keys)

EXAMPLE (ENABLE SSE-S3)

```
aws s3 cp file.txt s3://my-bucket/ \
--sse AES256
```

5. ACCESS CONTROL

- Control access using:

- ☒ IAM Policies
- ☒ Bucket Policies
- ☒ Access Control Lists (ACLs)
- ☒ Pre-signed URLs
- ☒ VPC Endpoint (Private Access)



BEST PRACTICE

Use least privilege access. Give only what is required, nothing more.

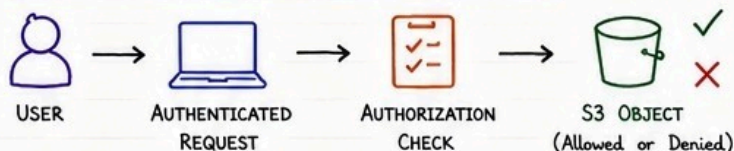


COMMON MISTAKE

Do not make an S3 bucket public just because an application needs access to an object. ❌

Instead, use IAM roles, Pre-signed URLs, or CloudFront signed URLs to provide access securely.

ACCESS FLOW (PRIVATE BY DEFAULT MINDSET)



EXAMPLE: PRE-SIGNED URL (TEMPORARY SECURE ACCESS)

- Generate a time-limited URL that gives access to an object.
- Useful for sharing a file securely without making the bucket public.

```
aws s3 presign s3://my-bucket/report.pdf --expires-in 3600
```

This URL can be shared and is valid for 1 hour. ⌚



REMEMBER

Security in S3 is all about controlling identity (who) and permissions (what). Use IAM + Bucket Policies + Encryption to keep your data safe.



PRIVATE BY DEFAULT MINDSET

- ☒ Everything is PRIVATE unless intentionally opened.
- ☒ Use authentication + authorization for every access.
- ☒ Monitor access using CloudTrail and S3 logs.
- ☒ Regularly review permissions and policies.

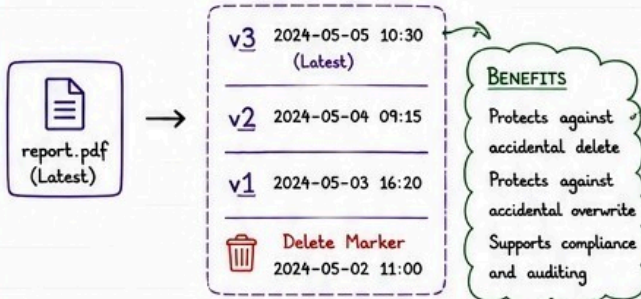


S3 DATA MANAGEMENT (ADVANCED FEATURES)

S3 helps you protect, automate and distribute your data.

1. VERSIONING

Keeps multiple versions of an object in the same bucket.
Helps recover from **accidental overwrites** or **deletes**.



Enable versioning

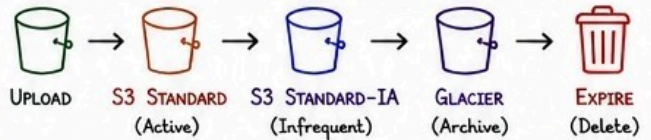
```
aws s3api put-bucket-versioning \
  --bucket my-bucket --versioning-
  configuration Status=Enabled
```

REMEMBER

Delete only adds a marker.
Old versions remain recoverable. 😊

2. LIFECYCLE RULES

Automatically transition objects between storage classes or delete them based on rules.



COMMON ACTIONS

- Transition → Move to cheaper storage class
- Expiration → Delete objects automatically

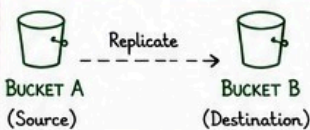
Example: Move objects to IA after 30 days
and delete after 365 days

```
aws s3api put-bucket-lifecycle-configuration \
  --bucket my-bucket --lifecycle-configuration file://lifecycle.json
```

3. REPLICATION

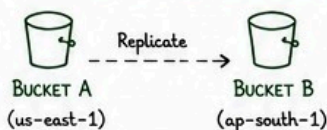
Automatically copy objects to another bucket.
Great for **DR**, **compliance** and **performance**.

SAME-REGION REPLICATION (SRR)



Use Case: Log aggregation, live copies, data processing pipelines.

CROSS-REGION REPLICATION (CRR)



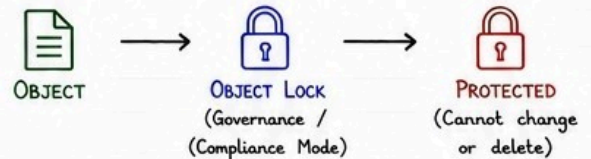
Use Case: Disaster recovery, data compliance, global access.

★ IMPORTANT NOTES

- Requires versioning (recommended).
- IAM permissions and replication role are required.
- Data transfer charges may apply.

4. OBJECT LOCK (WORM)

Helps protect objects from being deleted or overwritten for a fixed period. Useful for compliance.



MODES

- Governance Mode → Superusers can override.
- Compliance Mode → No one can override (even root).

★ KEY TAKEAWAY

Use these features together for a secure, cost-effective and highly available data strategy. ★

USEFUL AWS CLI COMMANDS

ACTION	COMMAND	DESCRIPTION
Enable Versioning	<code>aws s3api put-bucket-versioning --bucket my-bucket --versioning-configuration Status=Enabled</code>	Enables versioning on a bucket
List Versions	<code>aws s3api list-object-versions --bucket my-bucket --prefix report.pdf</code>	Shows all versions of an object
Delete a Specific Version	<code>aws s3api delete-object --bucket my-bucket --key report.pdf --version-id <version-id></code>	Deletes a specific version
Enable Replication	<code>aws s3api put-bucket-replication --bucket my-bucket --replication-configuration file://rep.json</code>	Configures replication
Put Lifecycle	<code>aws s3api put-bucket-lifecycle-configuration --bucket my-bucket --lifecycle-configuration file://lc.json</code>	Applies lifecycle rules

SAMPLE LIFECYCLE JSON (SNIPPET)

```
{
  "Rules": [
    {
      "ID": "MoveToIA",
      "Status": "Enabled",
      "Filter": { "Prefix": "Logs/" },
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        }
      ],
      "Expiration": { "Days": 365 }
    }
  ]
}
```

✓ BEST PRACTICES CHECKLIST

- | | |
|--|--|
| <ul style="list-style-type: none"> ✓ Enable versioning on critical buckets. ✓ Use lifecycle rules to reduce storage cost. ✓ Use replication for DR and compliance. ✓ Enable encryption (SSE-S3 or SSE-KMS). ✓ Monitor with CloudTrail and S3 metrics. | <ul style="list-style-type: none"> ✓ Use least privilege access. ✓ Enable Block Public Access. ✓ Use Object Lock for compliance. ✓ Regularly review and test backups. ✓ Tag buckets and objects for visibility. |
|--|--|

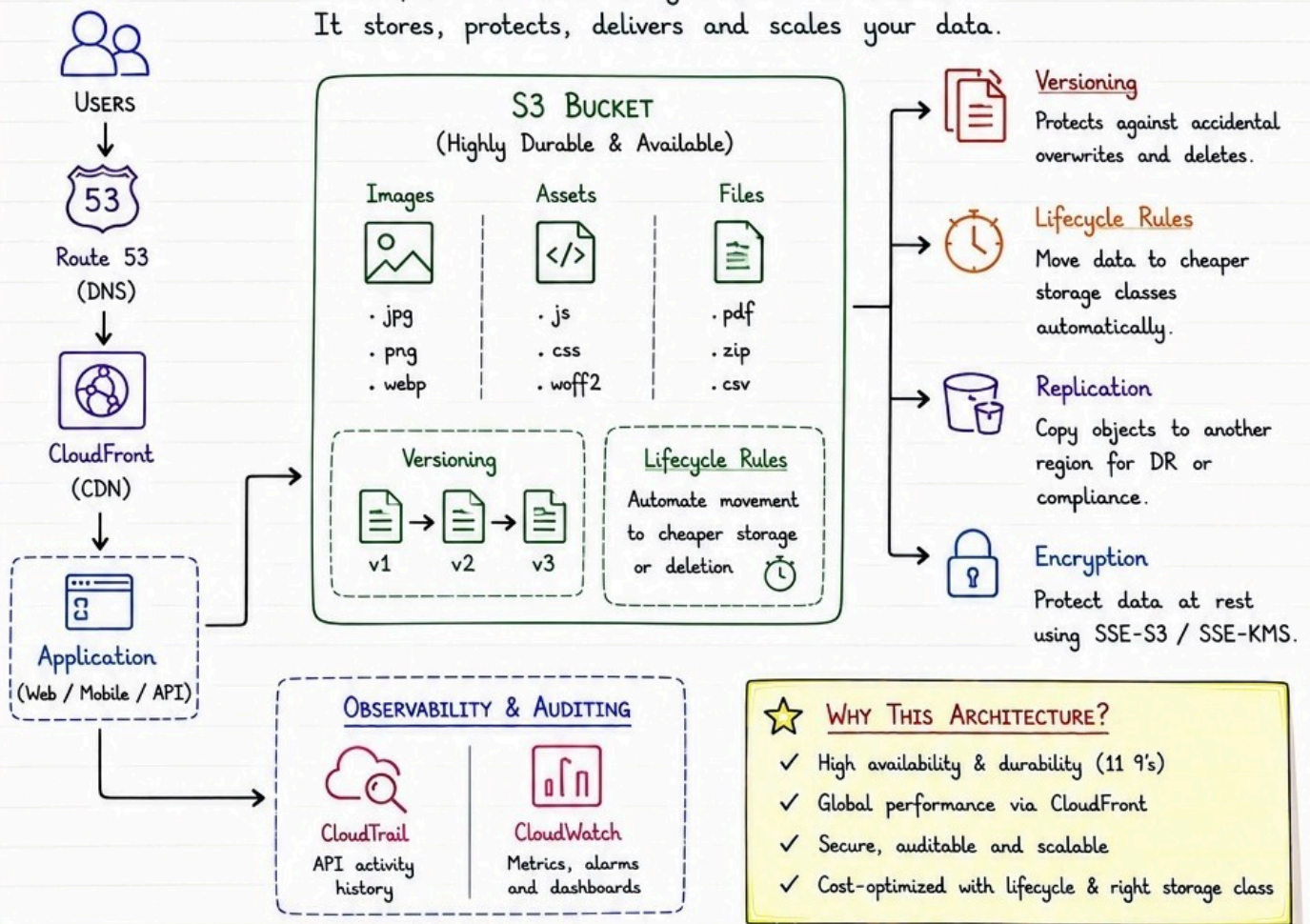
⚠ COMMON MISTAKES

- ✗ Not enabling versioning on important data.
- ✗ Not using lifecycle → higher storage cost.
- ✗ Storing everything in S3 Standard.
- ✗ Not enabling encryption.
- ✗ Misconfigured replication or permissions.

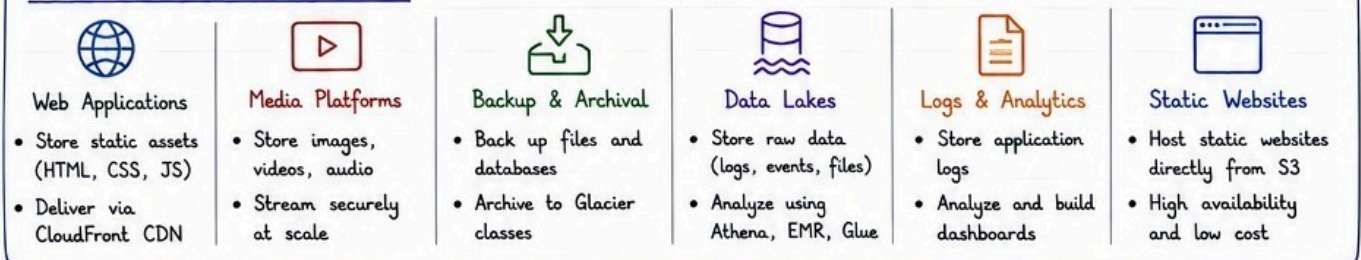
★ **REMEMBER** Versioning protects your data. Lifecycle saves your cost. Replication protects your business.

S3 IN A REAL PRODUCTION APPLICATION

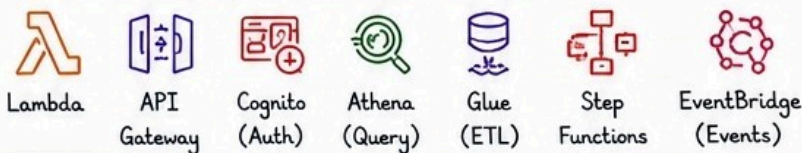
S3 fits into almost every modern architecture.
It stores, protects, delivers and scales your data.



COMMON REAL-WORLD USE CASES



INTEGRATION WITH OTHER AWS SERVICES



PRODUCTION MINDSET

Don't use EC2/EBS when your workload is fundamentally **OBJECT STORAGE**.

Let S3 handle scale, durability and availability.



ARCHITECTURE IN ACTION (EXAMPLE FLOW)

1. User requests → Route 53 resolves domain.
2. CloudFront serves cached content (if available).
3. On cache miss, CloudFront fetches from S3 bucket.
4. Application uploads new files to S3.
5. Lifecycle rules move old files to IA / Glacier.
6. CloudTrail logs every API call for audit.

\$ —

★ REMEMBER

- ✓ S3 stores your data.
- ✓ CloudFront delivers it fast.
- ✓ Versioning protects it.
- ✓ Lifecycle optimizes cost.
- ✓ Replication protects your business.
- ✓ CloudTrail monitors everything.



INTERVIEW TIP

When asked about S3 in production, talk about:

- Architecture
- Security
- Availability
- Cost Optimization
- Monitoring & Auditing

MAKE S3 PRODUCTION READY

1. COST OPTIMIZATION

S3 is cost-effective, but smart design makes it even better.



STORAGE COST

- Charged per GB/month
- Varies by storage class



REQUEST COST

- Charged per API request
- PUT, GET, LIST, etc.



DATA TRANSFER COST

- Outbound data is charged
- Inbound data is free



OTHER COST DRIVERS

- Storage class choice
- Lifecycle rules
- Data retrieval charges
- Cross-region replication



COST SAVING TIPS

- ✓ Use appropriate storage class
- ✓ Enable lifecycle policies
- ✓ Use Intelligent-Tiering for unknown patterns
- ✓ Delete unnecessary objects & versions
- ✓ Avoid frequent LIST operations
- ✓ Compress data before upload



2. BEST PRACTICES CHECKLIST

- ✓ Block unnecessary public access.
- ✓ Enable encryption (SSE-S3 / SSE-KMS).
- ✓ Enable versioning where needed.
- ✓ Use lifecycle policies to automate transitions and deletion.
- ✓ Choose the right storage class.
- ✓ Follow least privilege principle.
- ✓ Monitor access using CloudTrail & CloudWatch.
- ✓ Enable logging for audit and troubleshooting.
- ✓ Keep backups and enable replication.
- ✓ Tag buckets and objects for better management.



Secure
Smart
Scalable

PRO TIP

Good architecture + Good policies =
Secure + Scalable + Cost-optimized S3



3. QUICK HANDS-ON (AWS CLI)

① Create Bucket



② Upload Object



③ Set Permissions



④ Download Object



⑤ Enable Versioning



⑥ Configure Lifecycle



1) CREATE BUCKET

```
aws s3 mb s3://my-production-bucket \
--region ap-south-1
```

2) UPLOAD OBJECT

```
aws s3 cp file.txt s3://my-production-bucket/
aws s3 cp ./folder/ s3://my-production-bucket/ \
--recursive
```

3) SET PERMISSIONS (EXAMPLE: PRIVATE)

```
aws s3api put-public-access-block --bucket my-production-bucket \
--public-access-block-configuration \
{
  "BlockPublicAcls": true,
  "IgnorePublicAcls": true,
  "BlockPublicPolicy": true,
  "RestrictPublicBuckets": true
}
```

4) DOWNLOAD OBJECT

```
aws s3 cp s3://my-production-bucket/file.txt .
aws s3 sync s3://my-production-bucket ./download/
```

5) ENABLE VERSIONING

```
aws s3api put-bucket-versioning --bucket my-production-bucket \
--versioning-configuration Status=Enabled
```

6) CONFIGURE LIFECYCLE

```
aws s3api put-bucket-lifecycle-configuration --bucket \
my-production-bucket --lifecycle-configuration file://lifecycle.json
```

(Create lifecycle.json with rules for transitions + expiration)

LIFECYCLE EXAMPLE (VISUAL FLOW)



Sample Rule (lifecycle.json snippet)

```
{
  "Rules": [
    {
      "ID": "MoveToIA",
      "Status": "Enabled",
      "Transitions": [ { "Days": 30, "StorageClass": "STANDARD_IA" } ],
      "Expiration": { "Days": 365 }
    }
  ]
}
```

MONITOR & AUDIT



CloudTrail
API activity
history



CloudWatch
Metrics, alarms
& dashboards



S3 Server Access Logs
Request logs for
detailed analysis

Stay informed. Stay secure.

IMPORTANT NOTES



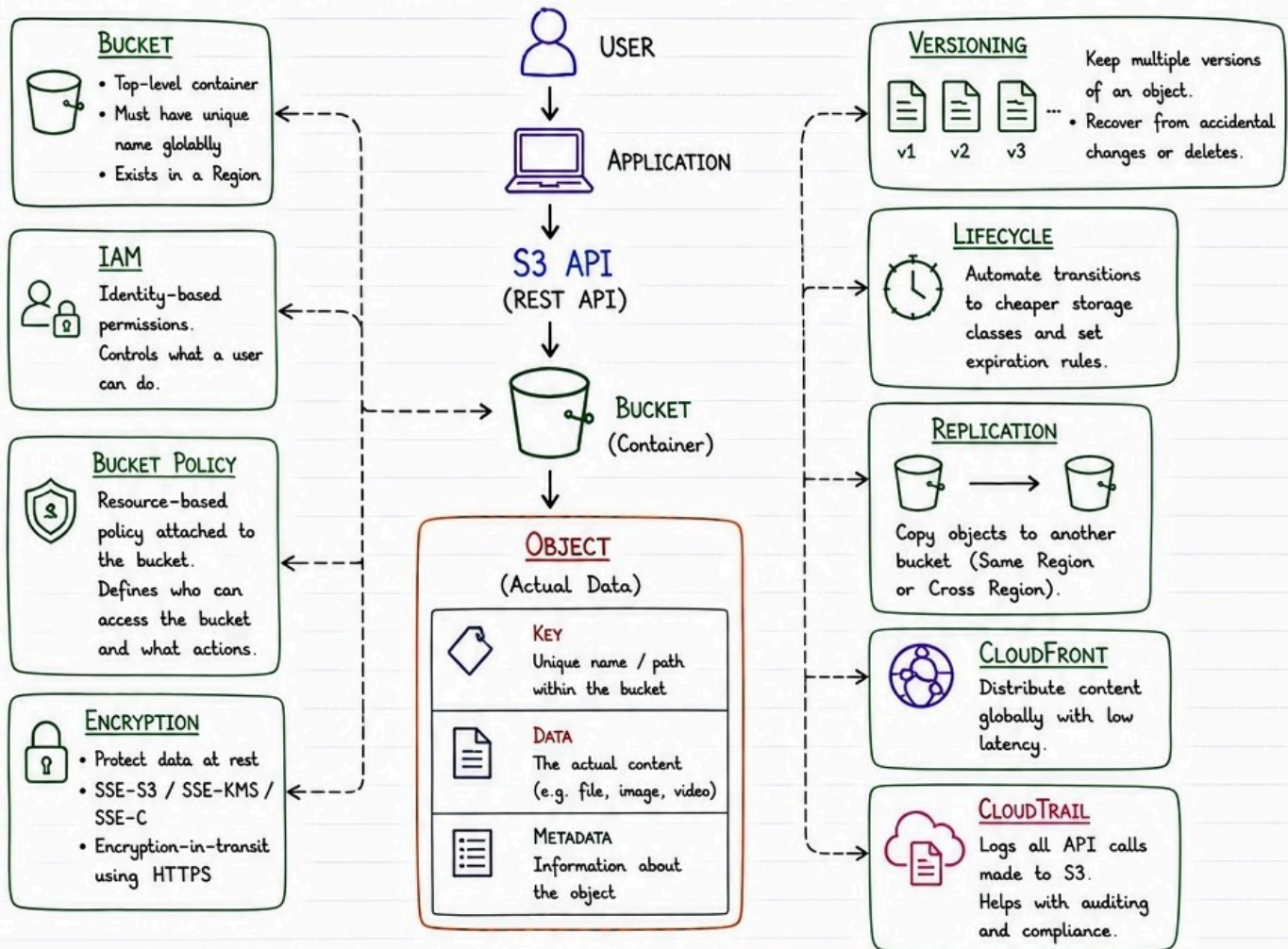
- Versioning increases storage usage.
- Glacier classes have retrieval charges.
- Deleting a delete marker doesn't restore the deleted object.
- Lifecycle rules apply to existing and new objects.
- Test rules in a non-production environment first.

REMEMBER

Automate. Monitor. Optimize.
That's how you run S3 in production.



S3 — COMPLETE MENTAL MODEL



S3 = DURABLE + AVAILABLE + SCALABLE + SECURE



11 9's
Durability



High
Availability



Massively
Scalable



Secure
by Design



10-SECOND INTERVIEW ANSWER

"Amazon S3 is a highly scalable object storage service used to store and retrieve data such as files, images, backups, logs and application assets. Data is stored as objects inside buckets."

10 S3 INTERVIEW QUESTIONS

1. What is Amazon S3?
2. What is an S3 bucket?
3. What is an S3 object?
4. What is an S3 object key?
5. S3 vs EBS?
6. What are S3 storage classes?
7. What is S3 Versioning?
8. What is an S3 Lifecycle Rule?
9. IAM Policy vs Bucket Policy?
10. How would you secure an S3 bucket?

MASTER S3

- | | |
|--------------------|---------------|
| ✓ Object Storage → | ✓ Replicate → |
| ✓ Secure → | ✓ Deliver → |
| ✓ Version → | ✓ Monitor → |
| ✓ Lifecycle → | ✓ Optimize → |

BONUS — QUICK CLI RECAP

```
aws s3 ls                                # List buckets
aws s3 ls s3://my-bucket                 # List objects
aws s3 cp file.txt s3://my-bucket/       # Upload file
aws s3 cp s3://my-bucket/file.txt .     # Download file
aws s3 sync ./folder s3://my-bucket/    # Sync folder
aws s3 rm s3://my-bucket/file.txt       # Delete object
aws s3 mb s3://my-bucket                # Create bucket
aws s3api put-bucket-versioning \
  --bucket my-bucket \                   # Enable versioning
  --versioning-configuration Status=Enabled
```



FINAL TAKEAWAY

S3 is the backbone of cloud storage. Master the concepts, use the right features, follow best practices and you will build scalable, secure and cost-effective solutions!

